

Отчет по финальному проекту курса DevOps Upgrade (5 поток)

студента Редкина Константина

номер студента s056635

1. Разработка схемы взаимодействия компонентов инфраструктуры и общее описание принципов развертывания решения

Для решения финального задания было принято использовать схему инфраструктуры с отдельными (приватными и публичными) сетями и размещаемыми в них виртуальными машин - нод кластера Kubernetes и виртуальных машин, предназначенных для служебных целей, непосредственно не входящих в функционал кластера.

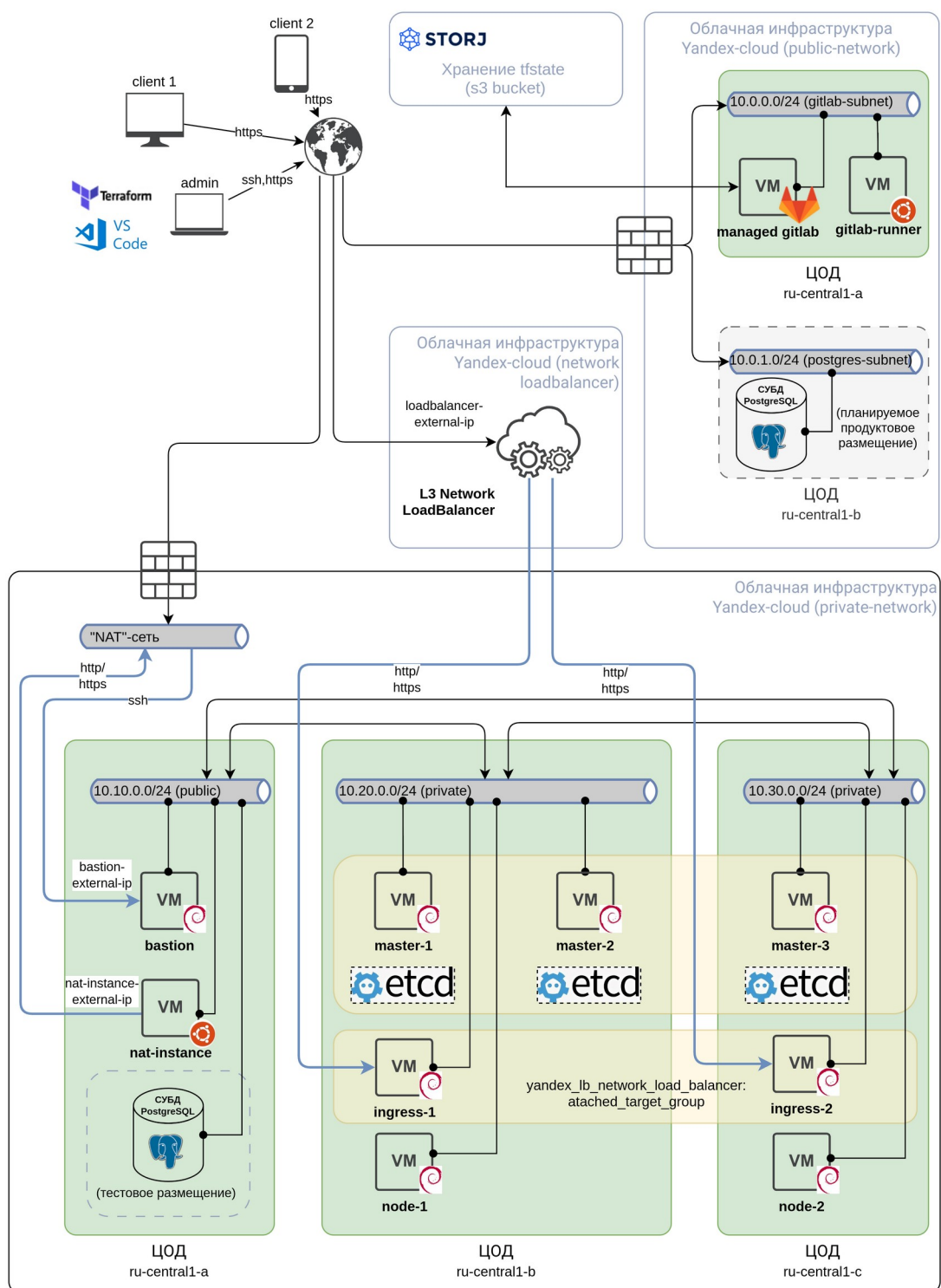


Рисунок 1 - Схема развертываемой инфраструктуры

Для возможности управлять кластером вместе с кластером через **terraform** поднимается виртуальная машина с именем **bastion**, с которой собственно и происходит запуск ansible-playbook **Kubespray** для развертывания кластера, и к которой в дальнейшем можно будет подключиться из-вне и поуправлять кластером Kubernetes с помощью пакетного менеджера **helm** или утилиты **kubectl**.

Также поднимается виртуальная машина **nat-instance** для возможности доступа нод кластера Kubernetes, в итоговом виде развертываемых в частных подсетях, во внешнюю сеть Интернет через нее.

Данная схема казалось более безопасной с точки зрения информационной безопасности (аналог sbx-хоста на учебных стендах Slurm), а также решала одну из проблем, полученных на раннем этапе выбора варианта решения задания - изначально инфраструктура в Yandex.Cloud развертывалась на триал-аккаунте, где было ограничение на количество внешних (nat) адресов к нодам и при развертывании кластера в схеме 3 master-ноды, 2 ingress-ноды и 3 worker-ноды появилось соответствующее уведомление.

После развертывания виртуальных машин формируется файл-инвентарь с адресами нод для будущего кластера Kubernetes, и затем посредством вызова terraform null_resource и remote-exec на машине **bastion** происходит запуск **Kubespray**. Развертывание кластера через связку **terraform+kubespray** для заранее определенных значений количества нод кластера хорошо выполнялось через сам terraform и подводных камней на горизонте не виделось.

Потому была попытка настроить автоматическое (не посредством CI/CD) обновление файла-инвентаря при масштабировании кластера с адресами добавляемых нод и последующей отработкой добавления worker-нод или их удаления, но при запуске terraform apply после изменения количества нод в переменных и последующего обновления файла-инвентаря через провайдер **local_file** выдавалась одиночная ошибка, показанная на рисунке 2. При этом процесс обновления кластера с новым количеством нод прерывался, но при повторном запуске terraform apply все применялось и новая нода успешно добавлялась в кластер.

На рисунка 3 и 4 эта же ошибка приведена в интерфейсе Gitlab CI при прогоне через первичный запуск при изменении количества нод уже развернутого кластера и при повторном перезапуске job-ы. Так как было непонятно, как данную ошибку исправлять и как в конечном итоге она в дальнейшем повлияет на процессы CI/CD и от идеи было решено отказаться, хотя на это было потрачено довольно много времени и в принципе решение казалось рабочим. Вот подобный issue на github <https://github.com/hashicorp/terraform-provider-local/issues/79> без адекватного (как написали **not very automation friendly**) решения.

```

local_file.kubespray_inventory: Destroying... [id=b3affac976f61ce9597c5bb20e7702853b264790]
local_file.kubespray_config: Destroying... [id=9c78168c7a65377eca394a0cc09f717b4aeb551f]
local_file.kubespray_inventory: Destruction complete after 0s
local_file.kubespray_config: Destruction complete after 0s
null_resource.kubespray_replacement_trigger: Destroying... [id=2736703838213338878]
null_resource.kubespray_replacement_trigger: Destruction complete after 0s
yandex_compute_instance_group.workers: Modifying... [id=cl160ibc3p6s0okuoalv]
yandex_compute_instance_group.workers: Still modifying... [id=cl160ibc3p6s0okuoalv, 10s elapsed]
yandex_compute_instance_group.workers: Still modifying... [id=cl160ibc3p6s0okuoalv, 20s elapsed]
yandex_compute_instance_group.workers: Still modifying... [id=cl160ibc3p6s0okuoalv, 30s elapsed]
yandex_compute_instance_group.workers: Still modifying... [id=cl160ibc3p6s0okuoalv, 40s elapsed]
yandex_compute_instance_group.workers: Still modifying... [id=cl160ibc3p6s0okuoalv, 50s elapsed]
yandex_compute_instance_group.workers: Modifications complete after 1m0s [id=cl160ibc3p6s0okuoalv]
null_resource.kubespray_replacement_trigger: Creating...
null_resource.kubespray_replacement_trigger: Creation complete after 0s [id=4097075916756266124]
local_file.kubespray_config: Creating...
local_file.kubespray_config: Creation complete after 0s [id=9c78168c7a65377eca394a0cc09f717b4aeb551f]

Error: Provider produced inconsistent final plan

When expanding the plan for local_file.kubespray_inventory to include new values learned so far during apply, provider
"registry.terraform.io/hashicorp/local" produced an invalid new value for .content: was ctty.StringVal("[all]\nmaster-1.s056635.local
ansible_host=10.20.0.31 ip=10.20.0.31\nmaster-2.s056635.local ansible_host=10.20.0.19 ip=10.20.0.19\nmaster-3.s056635.local ansible_host=10.30.0.31
ip=10.30.0.31\nnode-1.s056635.local ansible_host=10.20.0.9 ip=10.20.0.9\nnode-2.s056635.local ansible_host=10.30.0.32
ip=10.30.0.32\ningress-1.s056635.local ansible_host=10.20.0.6
ip=10.20.0.6\n\n[kube_control_plane]\nmaster-1.s056635.local\nmaster-2.s056635.local\nmaster-3.s056635.local\n\n[etcd]\nmaster-1.s056635.local\nmaster-2.s056635.local\nmaster-3.s056635.local\n\n[kube_node]\nnode-1.s056635.local\nnode-2.s056635.local\nnode-3.s056635.local\n\n[ingress-1.s056635.local\n\n[kube_ingress]\ningress-1.s056635.local\n\n[k8s_cluster:children]\nkube_node\nkube_control_plane\nkube_ingress\n"),
but now ctty.StringVal("[all]\nmaster-1.s056635.local ansible_host=10.20.0.31 ip=10.20.0.31\nmaster-2.s056635.local ansible_host=10.20.0.19
ip=10.20.0.19\nmaster-3.s056635.local ansible_host=10.30.0.31 ip=10.30.0.31\nnode-1.s056635.local ansible_host=10.20.0.9
ip=10.20.0.9\nnode-2.s056635.local ansible_host=10.30.0.32 ip=10.30.0.32\nnode-3.s056635.local ansible_host=10.20.0.11
ip=10.20.0.11\ningress-1.s056635.local ansible_host=10.20.0.6
ip=10.20.0.6\n\n[kube_control_plane]\nmaster-1.s056635.local\nmaster-2.s056635.local\nmaster-3.s056635.local\n\n[etcd]\nmaster-1.s056635.local\nmaster-2.s056635.local\nmaster-3.s056635.local\n\n[kube_node]\nnode-1.s056635.local\nnode-2.s056635.local\nnode-3.s056635.local\n\n[k8s_cluster:children]\nkube_node\nkube_control_plane\nkube_ingress\n").

This is a bug in the provider, which should be reported in the provider's own issue tracker.

```

Рисунок 2 - Ошибка при изменении количества нод на развернутом кластере

```

90 local_file.kubespray_config: Creation complete after 0s [id=9c78168c7a65377eca394a0cc09f717b4aeb551f]
91
92 Error: Provider produced inconsistent final plan
93
94 When expanding the plan for local_file.kubespray_inventory to include new
95 values learned so far during apply, provider
96 "registry.terraform.io/hashicorp/local" produced an invalid new value for
97 .content: was ctty.StringVal("[all]\nmaster-1.s056635.local
98 ansible_host=10.20.0.32 ip=10.20.0.32\nnode-1.s056635.local
99 ansible_host=10.20.0.4 ip=10.20.0.4\ningress-1.s056635.local
100 ansible_host=10.20.0.34 ip=10.20.0.34\ningress-2.s056635.local
101 ansible_host=10.30.0.25
102 ip=10.30.0.25\n\n[kube_control_plane]\nmaster-1.s056635.local\n\n[etcd]\nmaster-1.s056635.local\n
\n[kube_node]\nnode-1.s056635.local\nnode-2.s056635.local\nnode-3.s056635.local\n\n[kube_ingress]\ningress-1.s056635.local\nnode-2.s056635.local\nnode-3.s056635.local\n\n[k8s_cluster:children]\nkube
_node\nkube_control_plane\nkube_ingress\n"),
103 but now ctty.StringVal("[all]\nmaster-1.s056635.local
104 ansible_host=10.20.0.32 ip=10.20.0.32\nnode-1.s056635.local
105 ansible_host=10.20.0.4 ip=10.20.0.4\nnode-2.s056635.local
106 ansible_host=10.30.0.18 ip=10.30.0.18\ningress-1.s056635.local
107 ansible_host=10.20.0.34 ip=10.20.0.34\ningress-2.s056635.local
108 ansible_host=10.30.0.25
109 ip=10.30.0.25\n\n[kube_control_plane]\nmaster-1.s056635.local\n\n[etcd]\nmaster-1.s056635.local\n
\n[kube_node]\nnode-1.s056635.local\nnode-2.s056635.local\nnode-3.s056635.local\n\n[kube_ingress]\ningress-1.s056635.local\nnode-2.s056635.local\nnode-3.s056635.local\n\n[k8s_c
luster:children]\nkube_node\nkube_control_plane\nkube_ingress\n").
110
111 This is a bug in the provider, which should be reported in the provider's
112 own issue tracker.

```

Рисунок 3 - Первичный прогон CI/CD при масштабировании кластера

```

109 Terraform has been successfully initialized!
110 You may now begin working with Terraform. Try running "terraform plan" to see
111 any changes that are required for your infrastructure. All Terraform commands
112 should now work.
113 If you ever set or change modules or backend configuration for Terraform,
114 rerun this command to reinitialize your working directory. If you forget, other
115 commands will detect it and remind you to do so if necessary.
116 $ terraform apply --auto-approve plan
117
118 Error: Saved plan is stale
119
120 The given plan file can no longer be applied because the state was changed
121 by another operation after the plan was created.
122
123 Running after_script
124 Running after script...
125 $ export BASTION_IP=$(terraform output -raw bastion_external_ip)
126 $ echo BASTION_IP=${BASTION_IP} >> vars.env
127 Uploading artifacts for failed job
128 Uploading artifacts...
129 vars.env: found 1 matching artifact files and directories
130 Uploading artifacts as "dotenv" to coordinator... 201 Created id=135 responseStatus=201 Created tok
    en=64_e4spR
131 Cleaning up project directory and file based variables
132 ERROR: Job failed: exit code 1

```

Рисунок 4 - Вторичный перезапуск job-ы после ошибки с провайдером local_file

Потому решено было разворачивать через **terraform** только сами виртуальные машины и делать подготовку **bastion** к запуску ansible-скриптов (установка git и python3, скачивание kubespray из репозитория с github нужной версии и сама установка необходимых для работы ansible пакетов), а сам запуск **kubespray** вызывать следующими стейджами Gitlab CI посредством подключения к ВМ по SSH. Файл инвентаря планировал генерировать bash-скриптом, получая ip-адреса нод кластера после развертывания. После анализа вариантов было принято в качестве решения модифицировать исходный скрипт для **terraform**-модуля для **GCP** от разработчиков самого **kubespray** (скрипт доступен тут <https://github.com/kubernetes-sigs/kubespray/blob/master/contrib/terraform/gcp/generate-inventory.sh>). Модифицированный скрипт лежит в final/tf_infra/generate-inventory.sh и также работает на **terraform** ouptut и утилите **jq** и должен парсить адреса развернутых нод. На локальном компьютере, на котором установлен terraform и лежит проект, скрипт отработал отлично и сделал все, что нужно. Но при попытке запустить скрипт в Gitlab CI-пайплайне как **after_script** после работы **terraform apply** оказалось, что в официальном образе **hashicorp/terraform:latest** отсутствует утилита **jq**. А поставить ее туда не представляется возможным. Попытка установить terraform в контейнер alpine и запустить terraform init вызывало ошибку, представленную на рисунке 5, и пайплайн обрывался.

```
100 $ terraform init
101 Initializing the backend...
102
103 Error: Backend configuration changed
104
105 A change in the backend configuration has been detected, which may require
106 migrating existing state.
107
108 If you wish to attempt automatic migration of the state, use "terraform
109 init -migrate-state".
110 If you wish to store the current configuration with no changes to the
111 state, use "terraform init -reconfigure".
112
113 Cleaning up project directory and file based variables
114 ERROR: Job failed: exit code 1
```

Рисунок 5 — Ошибка команды terraform init в своем контейнере gitlab-раннера

Так как времени решить ошибку с **jq** и запуском **terraform** в своем контейнере уже не хватало, было решено в рамках целевой задачи запуска на прод приложения просто запускать кластер без дальнейшего изменения количества нод, оставив эту интересную задачу на дальнейшее самостоятельное изучение.

Также на инстансе **bastion** устанавливается **Docker** и скачивается образ с инструментом для графического управления кластером Kubernetes под названием **K9S**. Мне нравится этот инструмент, он легок в освоении, разворачивается быстро и представляет удобный инструментарий для получения информации об объектах кластера и их редактирования через встроенный редактор (обычно vim). На рисунке 6 приведен скриншот развернутых нод кластера с получаемыми метриками (CPU/MEM). Запускается после развертывания кластера и файла kube-конфига на **bastion** командой **sudo docker run --rm -it -v \$KUBECONFIG:/root/.kube/config derailed/k9s:latest**.

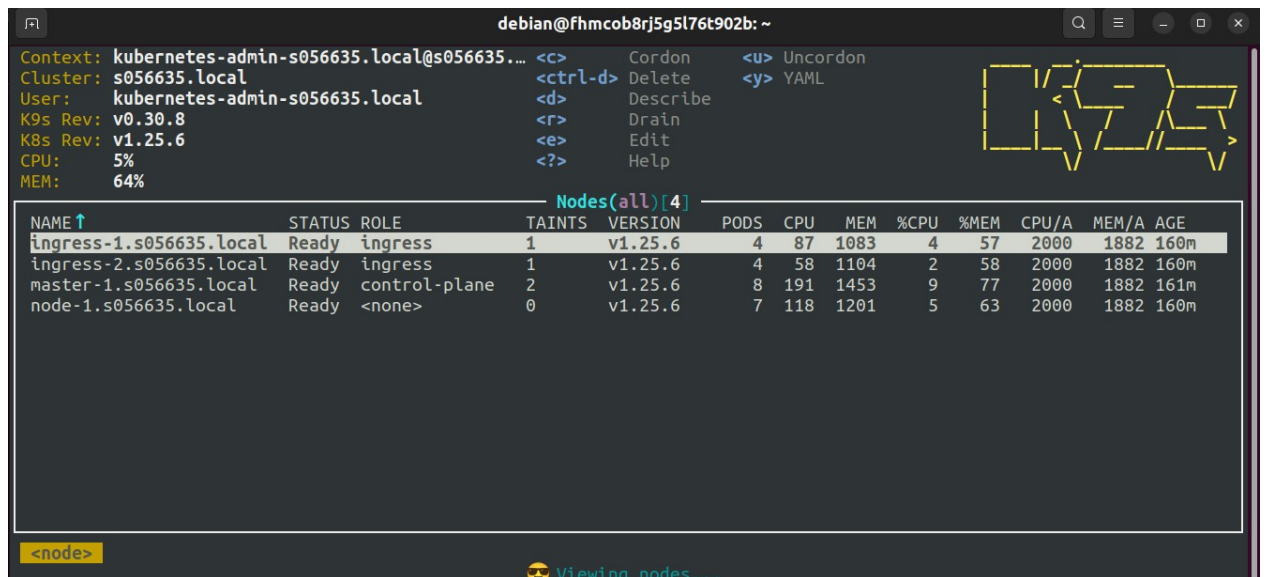


Рисунок 6 — Интерфейс инструмента управления кластером Kubernetes K9S

Отдельным проектом в Gitlab (final/postgres) также с помощью terraform разворачивается СУБД **Managed Service for PostgreSQL** (1 мастер-нода) с базой данных для запуска и работы приложения финального задания. Если разворачивать СУБД в сети, выделенной для нод Kubernetes, то СУБД будет зависеть от его инфраструктуры, что конечно нелогично. Логичным бы смотрелось решение сети кластера Kubernetes и СУБД разделить. Так как облачные сети при этом получились бы разные, то согласно документации Яндекс.Облака необходимо было бы включить публичный доступ для кластера **PostgreSQL** и настроить SSL для подключения к пулеру соединений для **PostgreSQL** от Яндекс.Облака «**Odyssey**». Но помимо этого нужно еще настроить маршрутизацию трафика к кластеру Kubernetes, а точнее к подам с приложением.

В «лабораторном» варианте разворачиваем **Managed Service for PostgreSQL** в сети кластера Kubernetes, но в подсети, имеющей публичный доступ, и к которой имеют доступ поды кластера. Как оказалось после нескольких попыток подключения к Managed Service for PostgreSQL, развернутому в таком варианте, настройка SSL также требуется.

Помимо настройки SSL оказалось, что в **Managed Service for PostgreSQL** во-первых разворачивается минимум 12 версии (в предложенном проекте mreferre/yelb был образ yelb-db с PostgreSQL 10 версии) и непонятно было как приложение себя поведет. И во-вторых, имя пользователя «**postgres**» предназначено для служебных целей и при создании этого пользователя появлялась ошибка, а в самом приложении я не увидел переменной для смены имени пользователя БД и при другом имени пользователя в логах **yelb-appserver** также появлялась ошибка). В-третьих, хоть и в переменных было указано имя хоста **Managed Service for PostgreSQL**, приложение продолжало упорно стучаться к хосту по имени **yelb-db**, что также приводило к нерабочему приложению (ошибка приведена на рисунке 7). В итоге СУБД **Managed**

Service for PostgreSQL разворачивается в ознакомительных целях, а работа в приложении с СУБД идет с запускаемым PostgreSQL 10 в кластере Кубернетес.

```
85.93.58.80 - - [10/Jan/2024:20:13:31 +0000] "GET /api/ihop HTTP/1.1" 500 30 0.0056
10.20.0.24 - - [10/Jan/2024:20:13:31 UTC] "GET /api/ihop HTTP/1.1" 500 30
https://s056635.redkin-edu.ru/ -> /api/ihop
2024-01-10 20:13:31 - PG::ConnectionBad - could not translate host name "yelb-db" to address: Name or service not known
:
/opt/bitnami/ruby/lib/ruby/gems/2.4.0/gems/pg-1.1.4/lib/pg.rb:56:in `initialize'
/opt/bitnami/ruby/lib/ruby/gems/2.4.0/gems/pg-1.1.4/lib/pg.rb:56:in `new'
/opt/bitnami/ruby/lib/ruby/gems/2.4.0/gems/pg-1.1.4/lib/pg.rb:56:in `connect'
/app/modules/restaurantsdbread.rb:17:in `restaurantsdbread'
/app/modules/getvotes.rb:5:in `getvotes'
/app/yelb-appserver.rb:114:in `block in <main>'
```

Рисунок 7 — Ошибка подключения к СУБД из контейнера приложения

Файл состояния всей инфраструктуры **terraform** terraform.tfstate хранится в s3-amazon-совместимом хранилище от провайдера **STORJ.io** (<https://storj.io/>). Данное решение обосновано наличием ограниченного бесплатного хранилища объемом до 150Гб, что в нашем варианте подходит более чем, и получаемой вследствие этого экономией ресурсов Яндекс.Облака и независимостью от статуса развернутого **Managed Service for GitLab** от Яндекс.Облака.

2. Подготовка Kubespray перед установкой кластера

Заполняем перед деплоем кластера файлы конфигурации Kubespray по примерам, показанных во время обучения, за исключением некоторых нюансов, связанных с выбором версии Kubespray (v1.22.1) и версии Кубернетес (v1.25.6). В качестве **CNI** выбран **Calico**, в качестве **CRI** — **containerd**. Часть параметров в конфиг будет вноситься при разворачивании (для примера заполнения некоторых параметров приведены установка домена и версии Кубернетес.

Для возможности запуска будущих приложений в режиме нескольких подов в автоматическом режиме при увеличении нагрузки (**Horisontal Pod Autoscaler**) для определения текущей нагрузки на поды приложения необходимо запустить сервер метрик. Metrics-server запустим через аддоны Kubespray. Также для возможности запуска helm-чартов установим менеджер **helm**. Данные настройки выполняем вручную перед запуском разворачивания кластера в файле проекта **tf_infra/kubespray_config/groups_var/k8s-cluster/addons.yml**.

```
# Metrics Server deployment
metrics_server_enabled: enabled
# metrics_server_container_port: 10250
metrics_server_kubelet_insecure_tls: true
metrics_server_metric_resolution: 15s
metrics_server_kubelet_preferred_address_types: "InternalIP,ExternalIP,Hostname"
metrics_server_host_network: true
# metrics_server_replicas: 1

# Helm deployment
helm_enabled: true
```


3. Развертывание Yandex Managed Service for GitLab

Для хранения кода и запуска автоматических пайплайнов CI/CD посредством веб-консоли Яндекс.Облака был развернут инстанс Yandex Managed Service for GitLab с адресом <https://s056635.gitlab.yandexcloud.net> в отдельной от кластера Kubernetes публичной сети и добавлены следующие проекты:

- **final/tf_infra** — проект инфраструктуры для кластера Kubernetes;
- **final/postgres** — проект с файлами terraform для развертывания инстанса PostgreSQL;
- **final/graduation_work** — проект с самим приложением и helm-чартами.

Для проверки со стороны slurm-support создан отдельный аккаунт со следующими учетными данными:

- User: slurm-io
- Email: slurmio@mail.swiftx.ru
- Password: igh-ya4A

Для запуска пайплайнов в проектах как минимум создадим переменные (Settings/CI/CD/Variables) для Gitlab CI для запуска/подключения в проектах к Яндекс.Облаку с их значениями:

```
YC_TOKEN=*****  
YC_CLOUD_ID=*****  
YC_FOLDER_ID=*****  
TF_VAR_folder_id=*****
```

4. Создание виртуальной машины с Docker для запуска Gitlab-runner и его регистрация

Для возможности запускать пайплайны в развернутом Yandex Managed Service for GitLab, а именно запуска проектов с непосредственным созданием всей необходимой инфраструктуры (кластера k8s и инстанса СУБД PostgreSQL), также в Яндекс.Облаке через веб-консоль создадим и запустим виртуальную машину с поддержкой контейнеризации на базе Docker со следующим файлом конфигурации docker-compose:

```
version: '3.7'  
services:  
  gitlab-runner:  
    container_name: gitlab-runner  
    image: gitlab/gitlab-runner:latest  
    restart: always  
    volumes:  
      - '/opt/gitlab-runner/data:/home/gitlab_ci_multi_runner/data'  
      - '/opt/gitlab-runner/config:/etc/gitlab-runner'  
      - '/var/run/docker.sock:/var/run/docker.sock:rw'  
    environment:  
      - CI_SERVER_URL=https://s056635.gitlab.yandexcloud.net/ci
```

После создания ВМ и запуска контейнера с образом `gitlab/gitlab-runner` зарегистрируем раннер в инстансе Gitlab через сгенерированную при создании раннера строку сервером Gitlab, подключившись к контейнеру (токен изменен):

```
gitlab-runner register --url https://s056635.gitlab.yandexcloud.net --token glrt-JWu7Mnh8cUE7pyK56qA9
```

с последующей правкой на виртуальной машине с gitlab-runner файла конфигурации `/opt/gitlab-runner/config/config.toml` и перезапуском соответствующего контейнера (если в этом есть необходимость):

```
concurrent = 1  
check_interval = 0  
shutdown_timeout = 0  
  
[session_server]  
session_timeout = 1800  
  
[[runners]]  
name = "s056635"  
url = "https://s056635.gitlab.yandexcloud.net"  
id = 3  
token = "glrt-JWu7Mnh8cUE7pyK56qA9"  
token_obtained_at = 2024-01-04T13:26:53Z  
token_expires_at = 0001-01-01T00:00:00Z  
executor = "docker"  
[runners.cache]  
MaxUploadedArchiveSize = 0  
[runners.docker]  
tls_verify = false  
image = "alpine:3.18"  
privileged = false  
disable_entrypoint_overwrite = false  
oom_kill_disable = false  
disable_cache = false  
volumes = ["/cache"]  
shm_size = 0  
network_mtu = 0
```

и убедимся, что gitlab-runner доступен для использования в пайплайнах по тегу с номером студента s056635 (как указали при регистрации shared-раннера):

☐	Status ?	Runner	Owner ?
☐	Online Idle	#3 (JWu7Mnh8c) Instance Version 16.7.0 🕒 Last contact: 1 minute ago 📍 158.160.36.126 🔄 0 👤 Created by k.redkin 11 minutes ago s056635	Administrator ✎ ⏸ ✖

Рисунок 8 — Проверка доступности gitlab-runner

Также для деплоя приложения в наш кластер Kubernetes нам потребуется либо настроить межсетевой доступ в кластер для развернутого выше раннера, либо развернуть раннер на bastion-хосте (docker executor) либо непосредственно в уже развернутом кластере Kubernetes (kubernetes executor). Развернем раннер в кластере Kubernetes через Helm. Получим токен для подключения и подготовим файл **values.yml** для запуска и регистрации раннера:

```
## The GitLab Server URL (with protocol) that want to register the runner against
## ref: https://docs.gitlab.com/runner/commands/README.html#gitlab-runner-register
##
gitlabUrl: https://s056635.gitlab.yandexcloud.net/

## The Registration Token for adding new Runners to the GitLab Server. This must
## be retrieved from your GitLab Instance.
## ref: https://docs.gitlab.com/ce/ci/runners/README.html
##
runnerRegistrationToken: glrt-3y6ZLSGoewbuTtzKB95V

runners:
  config: |
    [[runners]]
    name = "kubernetes"
    [runners.kubernetes]
    image = "alpine:3.17"
    cpu_request = "100m"
    cpu_limit = "500m"
    memory_request = "512Mi"
    memory_limit = "1024Mi"
  tags: kubernetes

## For RBAC support:
rbac:
  create: true

## Run the gitlab-bastion container with the ability to deploy/manage containers of jobs
## cluster-wide or only within namespace
clusterWideAccess: false

## Use the following Kubernetes Service Account name if RBAC is disabled in this Helm
chart (see rbac.create)
##
# serviceAccountName: gitlabrunnersa

## Use podSecurity Policy
## ref: https://kubernetes.io/docs/concepts/policy/pod-security-policy/
podSecurityPolicy:
  enabled: false
  resourceNames:
    - gitlab-runner

## Configure resource requests and limits
```

```
## ref: http://kubernetes.io/docs/user-guide/compute-resources/
##
resources:
  limits:
    memory: 256Mi
    cpu: 200m
  requests:
    memory: 128Mi
    cpu: 50m
```

И далее добавим helm-репозиторий gitlab и запустим раннер через подготовленный файл:

```
helm repo add gitlab https://charts.gitlab.io
helm repo update gitlab
helm upgrade -i gitlab-runner gitlab/gitlab-runner -f values.yml -n gitlab-runner --create-namespace
```

Затем также проверим доступность раннера из Gitlab:

☐	Status ?	Runner	Owner ?
☐	Online Idle	#7 (3y6ZLSGoe) 4 Instance Version 16.7.0 ⌚ Last contact: 15 minutes ago 158.160.117.233 (+3) 6 📅 Created by k.redkin 1 hour ago kubernetes	Administrator

Рисунок 9 — Проверка доступности gitlab-runner

5. Развертывание ingress-контроллера

Для запуска ingress-контроллера на нодах, определенных для ingress, перед развертыванием кластера добавляем метку ingress посредством создания дополнительного файла **kube_ingress.yml** в папке **tf_infra/kubespray_config/group_vars** со следующим содержимым:

```
node_labels:  
node-role.kubernetes.io/ingress: ""
```

либо уже после создания кластера присваиваем эту метку ingress-нодам командой:

```
kubectl label node ingress-1.s056635.local node-role.kubernetes.io/ingress=true
```

Также через файл **tf_infra/kubespray_config/group_vars/kube_ingress.yml** запретим развертывать на ingress-ноде пользовательские поды, установив соответствующий **taint**.

```
node_taints:  
- "node-role.kubernetes.io/ingress=:NoSchedule"
```

После того как кластер Kubernetes будет развернут добавляем официальный helm-репозиторий **ingress-nginx** в доступные и обновляем пакеты менеджера **Helm**:

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx  
helm repo update
```

Создадим в кластере файл со значениями переменных для запуска ingress-контроллера на указанных нодах **nginx-ingress-values.yml** со следующим содержимым:

```
controller:  
kind: DaemonSet  
ingressClassResource:  
default: true  
watchIngressWithoutClass: true  
nodeSelector:  
node-role.kubernetes.io/ingress: ""  
hostNetwork: true  
tolerations:  
- key: node-role.kubernetes.io/ingress  
operator: "Exists"  
service:  
type: LoadBalancer  
nodePorts:  
http: 30080  
https: 30443
```

после чего устанавливаем ingress-контроллер в отдельный неймспейс:

```
helm install ingress-nginx ingress-nginx/ingress-nginx -n ingress-nginx --create-namespace  
--values nginx-ingress-values.yml
```


и проверяем статус подов развернутого контроллера:

Pods(ingress-nginx)[2]												
NAME ↑	PF	READY	STATUS	RESTARTS	CPU	MEM	%CPU/R	%CPU/L	%MEM/R	%MEM/L	IP	NODE
ingress-nginx-controller-c47p7	●	1/1	Running	0	2	67	2	n/a	75	n/a	10.30.0.12	ingress-2.
ingress-nginx-controller-kq4nr	●	1/1	Running	0	1	66	1	n/a	73	n/a	10.20.0.35	ingress-1.

Рисунок 10 — Проверка статуса подов ingress-контроллера, развернутого в режиме Daemonset

6. Развертывание cert-manager

Для работы нашего приложения по https развернем в кластере Cert-manager. Для начала создадим файл манифеста для ClusterIssuer — letsencrypt-staging для выпуска тестовых сертификатов от ЦА Let'sEncrypt и letsencrypt-production для получения продуктовых сертификатов:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-staging
spec:
  acme:
    server: https://acme-staging-v02.api.letsencrypt.org/directory
    email: k.redkin@swiftx.ru
    privateKeySecretRef:
      name: letsencrypt-staging
    solvers:
      - http01:
          ingress:
            class: nginx
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-production
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: k.redkin@swiftx.ru
    privateKeySecretRef:
      name: letsencrypt-production
    solvers:
      - http01:
          ingress:
            class: nginx
```

Затем установим в кластере для cert-manager CRDs, развернем сам cert-manager в неймспейсе **cert-manager** через helm-чарт и запустим созданные манифесты:

```
kubectl apply -f https://github.com/jetstack/cert-manager/releases/download/v1.12.3/cert-manager.crds.yaml
```

```
kubectrl create namespace cert-manager
```

```
helm repo add jetstack https://charts.jetstack.io  
helm repo update
```

```
helm install cert-manager \  
--namespace cert-manager \  
--version v1.12.3 \  
--set ingressShim.defaultIssuerName=letsencrypt-staging \  
--set ingressShim.defaultIssuerKind=ClusterIssuer \  
jetstack/cert-manager
```

```
kubectrl apply -f cert-manager-values.yml
```

7. Установка gitlab-agent

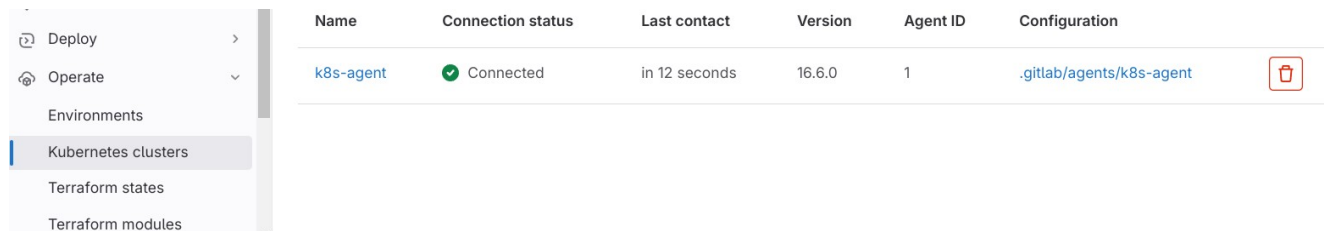
Для возможности деплоить обновления приложения через Helm в кластер Kubernetes посредством Gitlab CI установим **gitlab-agent**. Для этого сформируем в проекте с приложением файл-конфиг **final/graduation_work/.gitlab/k8s-agent/config.yaml** со следующим содержимым:

```
ci_access:  
projects:  
- id: final/graduation_work  
groups:  
- id: final
```

В принципе можно ограничиться разделом **groups**, но для верности пропишем путь до нашего проекта с приложением. Затем в разделе **Gitlab/Operate/Kubernetes clusters** нашего проекта создадим подключение, выбрав имя **k8s-agent**. Далее Gitlab нам сформирует команду для запуска агента в кластере через менеджер **Helm** в следующем формате (токен изменен):

```
helm upgrade --install k8s-agent gitlab/gitlab-agent --namespace study-project --create-  
namespace --set image.tag=v16.6.0 --set config.token=glagent-  
kgVLjtJm9_iPh2x8bvNdddCrNxX1jsB625W3HjXrX_tzhNtfAQ --set  
config.kasAddress=wss://s056635.gitlab.yandexcloud.net/-/kubernetes-agent/
```

После разворачивания мы должны увидеть подключение в интерфейсе Gitlab и его статус:



Name	Connection status	Last contact	Version	Agent ID	Configuration
k8s-agent	Connected	in 12 seconds	16.6.0	1	.gitlab/agents/k8s-agent

Рисунок 11 — Статус подключения gitlab-агента

После чего можно приступать к настройке деплоя непосредственно самого приложения в кластер.

8. Подготовка ноды node-1 для локального хранения данных подов

Перед запуском приложения в развернутом контуре на рабочей ноде «node-1» необходимо создать StorageClass с типом local-storage для размещения на нем данных подов с кеширующей СУБД Redis и СУБД PostgreSQL, если применяется внутренняя БД (в качестве предварительных тестовых развертываний приложения, и как оказалось в конце в продуктивном варианте). Это можно также сделать как через addons.yml в папке group_vars плейбука kubespray, так и вручную. Для разнообразия сделаем это вручную после запуска кластера. Также необходимо перед созданием PV создать указанные директории на самой ноде «node-1» командой:

```
sudo mkdir /mnt/pv1 /mnt/pv2
```

Текст манифеста **storage.yml** для k8s создания StorageClass и PersistentVolume:

```
apiVersion: storage.k8s.io/v1  
kind: StorageClass  
metadata:  
  name: local-storage  
provisioner: kubernetes.io/no-provisioner  
volumeBindingMode: WaitForFirstConsumer
```

```
apiVersion: v1  
kind: PersistentVolume  
metadata:  
  name: pv1  
spec:  
  capacity:  
    storage: 1Gi  
accessModes:  
  - ReadWriteOnce  
persistentVolumeReclaimPolicy: Retain  
storageClassName: local-storage  
local:  
  path: /mnt/pv1  
nodeAffinity:  
  required:  
    nodeSelectorTerms:  
    - matchExpressions:  
      - key: kubernetes.io/hostname  
        operator: In  
        values:  
        - node-1.s056635.local
```

```
apiVersion: v1  
kind: PersistentVolume  
metadata:
```

name: pv2
spec:
 capacity:
 storage: 1Gi
 accessModes:
 - **ReadWriteOnce**
 persistentVolumeReclaimPolicy: Retain
 storageClassName: local-storage
 local:
 path: /mnt/pv2
 nodeAffinity:
 required:
 nodeSelectorTerms:
 - **matchExpressions:**
 - **key: kubernetes.io/hostname**
 operator: In
 values:
 - **node-1.s056635.local**

В принципе все необходимые манифесты лежат в проекте `finar/tf_infra/manifests` и загружаются на ноду `bastion` в папку пользователя `debian`.

9. Запуск пайплайнов CI/CD и проверка работы приложения

Запуск всех пайплайнов по разворачиванию инфраструктуры происходит при коммите в ветку main проектов.

9.1 Описание работы пайплайна для проекта инфраструктуры (final/tf_infra)

В принципе общий алгоритм работы пайплайна по разворачиванию инфраструктуры описан в разделах 1 и 2. При разворачивании кластера необходимо установить значение переменной KUBESPRAY_ACTION в значение «create_cluster», а в файле terraform.tfvars выставить количество нод мастер-группы, ingress-группы и рабочих нод и сделать коммит в ветку main.

Запустится сначала этап проверки линтерами terraform, затем генерация файла с планом разворачивания и последним этап вручную запускается для данного плана разворачивание виртуальных машин и затем сама установка кластере Kubernetes.

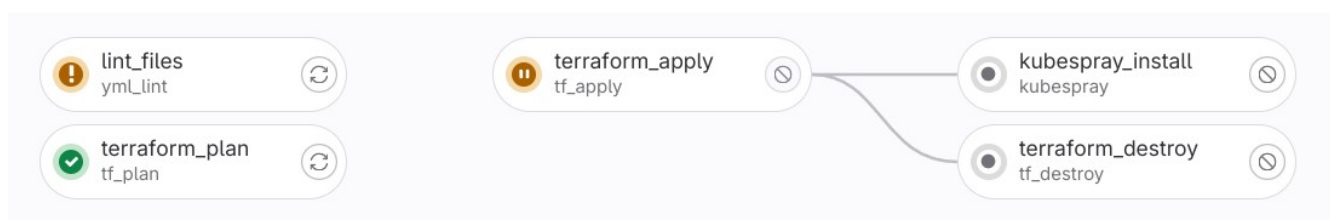


Рисунок 12 — Порядок работы стейджей пайплайна с инфраструктурой

На выходе пайплайна (в данном случае стейдж terraform_apply) имеем файл с приватным ключом подключения к экземпляру bastion и вывод terraform output с необходимой информацией. Отдельно стейдж kubespray_install не запускаем (он не работоспособен из-за нерабочего скрипта получения адресов нод для создания файла инвентаря и дальнейшего разворачивания кластера).

После разворачивания необходимо прописать у регистратора домена redkin-edu.ru dns-запись s056635.redkin-edu.ru для полученного ip-адреса сетевого балансировщика, либо в качестве локальной проверки через правку файла /etc/hosts (для Linux-систем) и C:\Windows\System32\drivers\etc\hosts (для Win-систем) добавлением записи с ip-адресом и dns-именем сервиса с приложением.

Удаление инфраструктуры выполняется стейджем terraform_destroy.

9.2 Описание пайплайна для СУБД PostgreSQL (final/postgres)

Вобщем-то пайплайн схож с пайплайном инфраструктуры — разворачивание происходит при коммите в main-ветку проекта.

Заполняем необходимые данные в `terraform.tfvars`, в том числе данные учетной записи пользователя и пароля. Пока они хранятся напрямую в проекте, в планах было конечно же их перенести в переменные Gitlab CI, но не успел.

После коммита запустится этап проверки линтерами `terraform`, затем генерация файла с планом развертывания и последним этап вручную запускается для данного плана развертывание инстанса `Postgres` в указанной в файле `network.tf` сети/подсети.

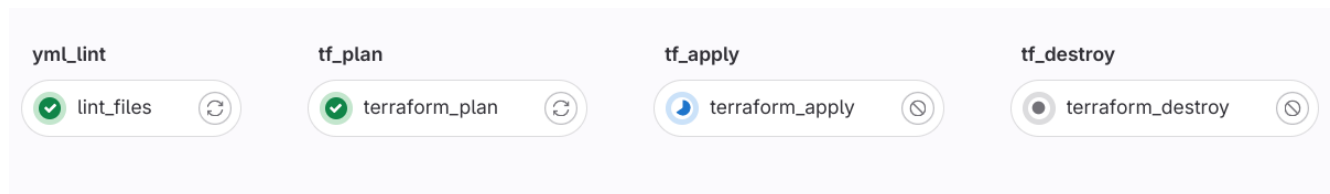


Рисунок 13 — Порядок работы стейджей пайплайна с инстансом PostgreSQL

Удаление инфраструктуры выполняется стейджем `terraform_destroy`.

8.3 Описание пайплайна для приложения

Поставка приложения в кластер реализована через `helm`-чарт, размещенные в папке `.helm`. Запуск пайплайна также как и в предыдущих вариантах осуществляется при коммите в `main`-ветку проекта.

Заполняем значения переменных в файле `.helm/values.yml` и делаем коммит.

После коммита запустится этап проверки линтерами `helm` с выводом сгенерированных согласно заданных переменных шаблонов. Затем на этапе `deploy` осуществляется непосредственная поставка приложения в кластер посредством `gitlab-agent`, развернутого в том же неймспейсе проекта (в данном случае `study-project`). После успешного этапе деплоя приложение через некоторое время должно запуститься по адресу <https://s056635.redkin-edu.ru>, что как раз проверяется на следующем этапе `validate` (через образ с установленным `curl`).

Также предусмотрен ручной роллбек приложения на предыдущую версию на последнем этапе.

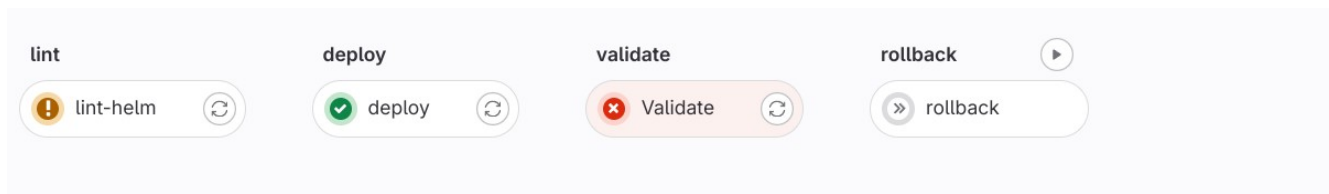


Рисунок 14 — Порядок работы стейджей пайплайна с поставкой кода приложения в кластер

В зависимости от выбранных настроек можно выбрать использование внешнего инстанса СУБД PostgreSQL, либо внутреннего, развертываемого в поде в том же неймспейсе, что и само приложение. Для подготовки базы в поде с приложением запускаются init-контейнеры, которые в зависимости от выбора типа используемой СУБД (внешний в Яндекс.Облаке или внутренний в кластере Kubernetes), запускают init-скрипт. В случае с внешней СУБД перед запуском init-скрипта производится скачивание корневого сертификата Яндекс для настройки SSL-подключения к PostgreSQL и копирование его в следующий init-контейнер для осуществления «миграций», а затем и в рабочий контейнер с backend-приложением (yelp-appserver).

Для кеша приложения также внутри кластера Kubernetes поднимается нереляционная СУБД Redis, но в целом в зависимости от выбора параметров в values.yml возможно также использовать внешний инстанс Redis.

Внутренние СУБД Redis и PostgreSQL могут быть развернуты с параметром Persistent Volume. Для примера развертывание при выборе параметра «persistentVolume: enabled» в обоих СУБД происходит с характеристиками 1GB и Storage-класс «local-storage» с хранением данных на ноде node-1 (настройку делали в разделе 8).

Выпуск ssl-сертификата для приложения осуществляется при запуске ресурса ingress к приложению посредством cert-manager, развертываемого для выпуска как тестовых, так и продуктовых сертификатов. Выбор типа ClusterIssuer осуществляется посредством аннотации к ресурсу ingress.

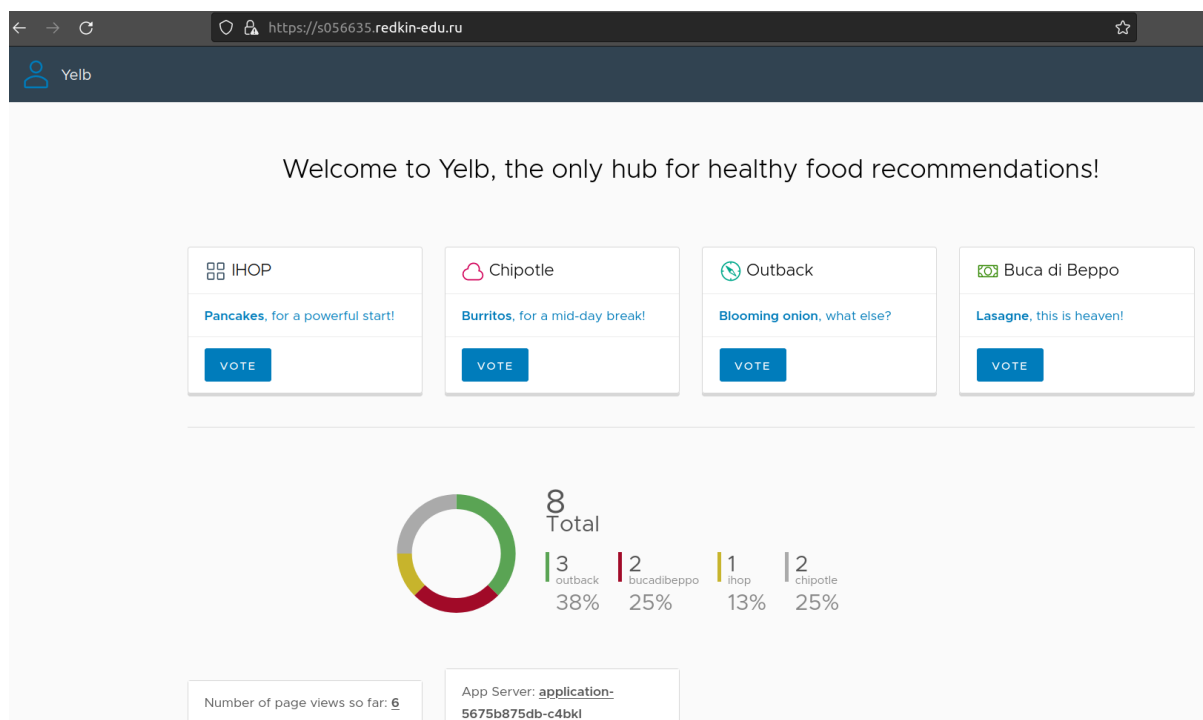


Рисунок 15 — Результат развертывания приложения

10. Встроенные дашборды Яндекс.Облака

Не успел разобраться, что можно с этими дашбордами делать, но приведу пару скриншотов:

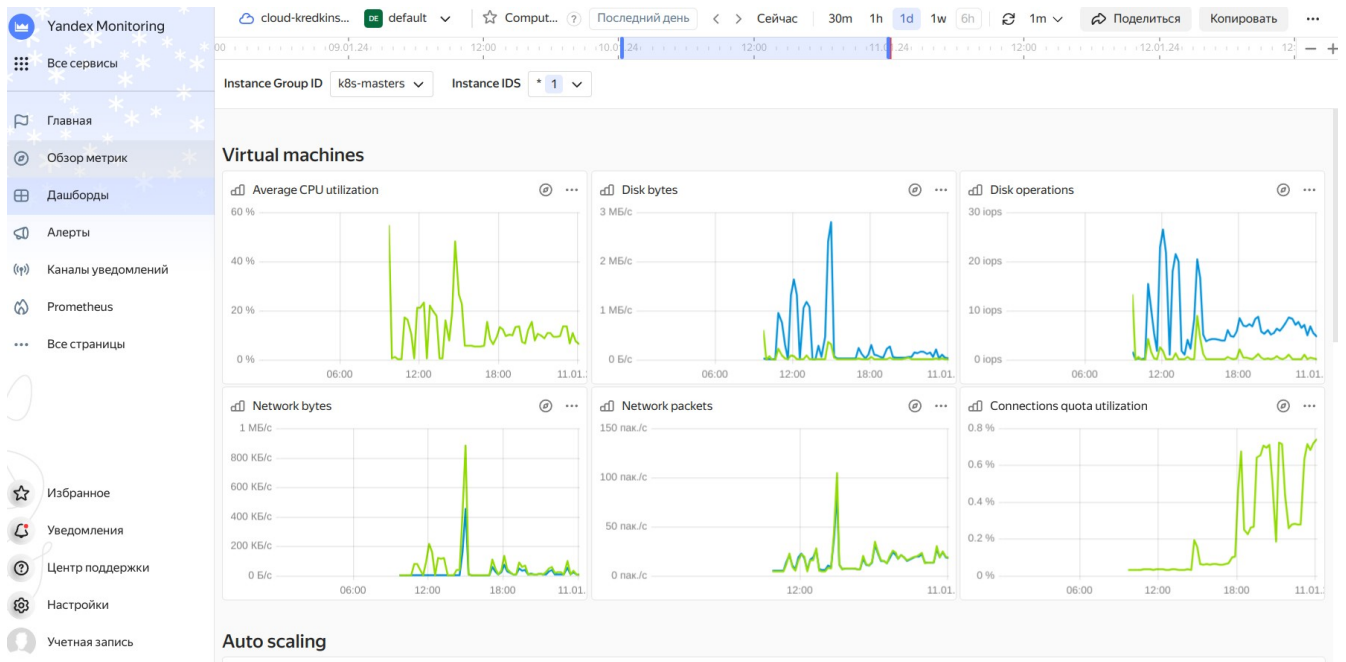


Рисунок 16 — Дашборды ресурсов для группы мастер-нод за последний день

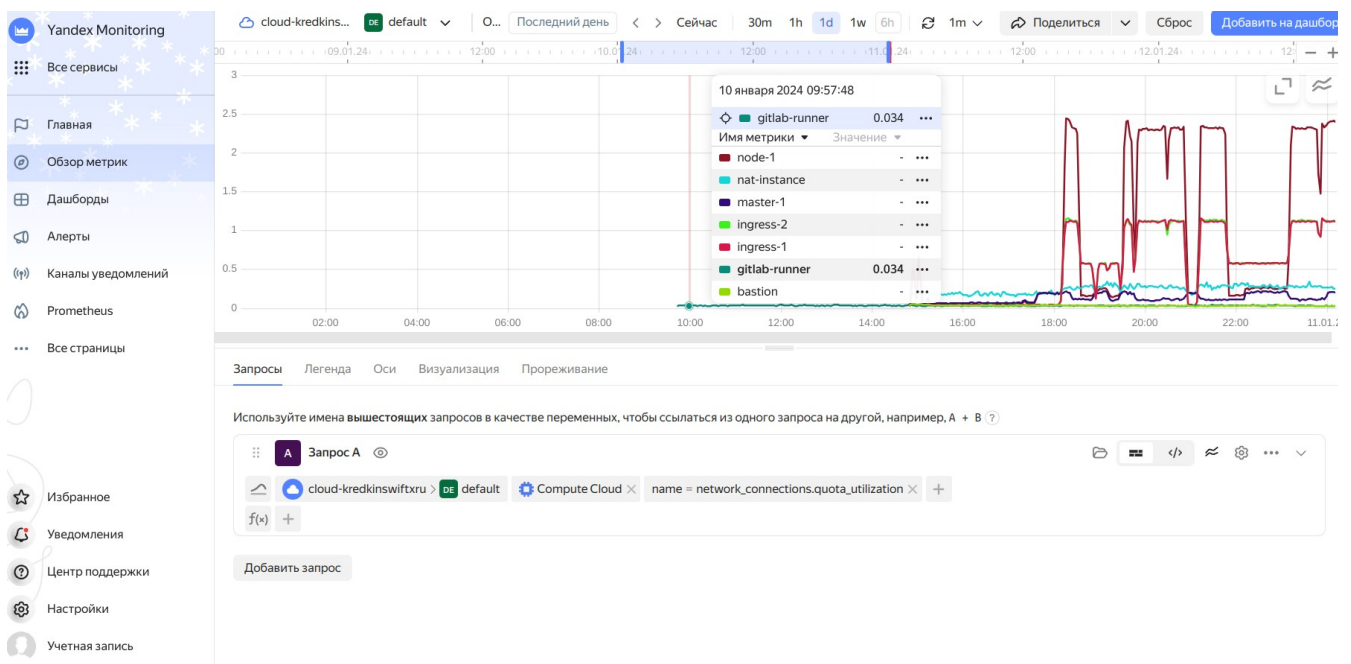


Рисунок 17 — Метрики сетевых соединений для группы Compute Cloud